

Machine Learning: Exponential Family and Generalized Linear Models (GLMs)

Quoc Kien

1. Họ Phân Phối Số Mũ (Exponential Family Distribution - EFD)

Một phân phối xác suất đơn biến thuộc **Họ phân phối số mũ** nếu hàm mật độ xác suất (hoặc hàm khối xác suất) của nó có thể biến đổi về dạng chuẩn tắc sau:

$$P(y | \eta) = b(y) \exp(\eta^T T(y) - a(\eta))$$

Trong đó: * y : Biến ngẫu nhiên của dữ liệu quan sát. * η : Tham số tự nhiên (Natural parameter / Canonical parameter). * $T(y)$: Thống kê đủ (Sufficient statistic) — thông thường trong các bài toán cơ bản $T(y) = y$. * $b(y)$: Thuộc đo cơ sở (Base measure). * $a(\eta)$: Hàm chuẩn hóa log (Log-partition function), đóng vai trò như một hằng số chuẩn hóa để đảm bảo tổng tích phân xác suất bằng 1.

Các tính chất toán học cốt lõi (Properties)

Hàm log-partition $a(\eta)$ mang thông tin cực kỳ quan trọng về các mô-men của phân phối: 1. **Kỳ vọng (Mean)**: Đạo hàm bậc nhất của $a(\eta)$ chính là kỳ vọng của thống kê đủ:

$$\mathbb{E}[y | \eta] = \frac{\partial a(\eta)}{\partial \eta}$$

2. **Phương sai (Variance)**: Đạo hàm bậc hai của $a(\eta)$ chính là phương sai của thống kê đủ:

$$\text{Var}(y | \eta) = \frac{\partial^2 a(\eta)}{\partial \eta^2}$$

2. Chuẩn hóa các phân phối kinh điển về dạng EFD

2.1 Phân phối Bernoulli (Nền tảng của Logistic Regression)

Phân phối Bernoulli mô hình hóa biến ngẫu nhiên nhị phân $y \in \{0, 1\}$ với xác suất thành công $P(y = 1) = \phi$:

$$P(y | \phi) = \phi^y (1 - \phi)^{1-y}$$

Sử dụng phép biến đổi mũ và logarit ($\exp(\log(\cdot))$) để đưa về dạng chuẩn tắc:

$$P(y | \phi) = \exp(\log(\phi^y (1 - \phi)^{1-y}))$$

$$P(y | \phi) = \exp(y \log \phi + (1 - y) \log(1 - \phi))$$

$$P(y | \phi) = \exp(y \log \phi + \log(1 - \phi) - y \log(1 - \phi))$$

$$P(y | \phi) = \exp\left(y \log\left(\frac{\phi}{1 - \phi}\right) + \log(1 - \phi)\right)$$

Đối chiếu với phương trình tổng quát của Họ phân phối số mũ, ta rút ra các thành phần chuẩn tắc: * **Base measure:** $b(y) = 1$ * **Sufficient statistic:** $T(y) = y$ * **Tham số tự nhiên η :**

$$\eta = \log\left(\frac{\phi}{1 - \phi}\right)$$

Nếu ta giải phương trình này để tìm ϕ theo η :

$$e^\eta = \frac{\phi}{1 - \phi} \implies e^{-\eta} = \frac{1 - \phi}{\phi} = \frac{1}{\phi} - 1$$

$$\phi = \frac{1}{1 + e^{-\eta}} = \sigma(\eta)$$

(Đây là minh chứng tối thượng vì sao hàm số Sigmoid lại xuất hiện một cách tự nhiên trong mô hình Logistic Regression). * **Log-partition function $a(\eta)$:**

$$a(\eta) = -\log(1 - \phi) = -\log\left(1 - \frac{1}{1 + e^{-\eta}}\right) = \log(1 + e^\eta)$$

2.2 Phân phối Gaussian (Nền tảng của Linear Regression)

Xét phân phối chuẩn Gaussian với phương sai cố định $\sigma^2 = 1$ để đơn giản hóa biểu thức đại số:

$$P(y | \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y - \mu)^2\right)$$

Khai triển hằng đẳng thức đáng nhớ bên trong hàm mũ:

$$P(y | \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y^2 - 2y\mu + \mu^2)\right)$$

$$P(y | \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right) \exp\left(y\mu - \frac{1}{2}\mu^2\right)$$

Đối chiếu với cấu trúc dạng chuẩn tắc EFD, ta phân rã được các thành phần: * **Base measure:** $b(y) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right)$ * **Sufficient statistic:** $T(y) = y$ * **Tham số tự nhiên η :** $\eta = \mu$ * **Log-partition function $a(\eta)$:** $a(\eta) = \frac{1}{2}\mu^2 = \frac{1}{2}\eta^2$

3. Bản chất Tính Lỗi của bài toán MLE thuộc họ EFD

Xét tập dữ liệu gồm n mẫu độc lập i.i.d. Hàm Log-Likelihood theo tham số tự nhiên η là:

$$\ell(\eta) = \sum_{i=1}^n \log P(y^{(i)} | \eta) = \sum_{i=1}^n [\log b(y^{(i)}) + \eta^T T(y^{(i)}) - a(\eta)]$$

Để tìm điểm tối ưu, ta lấy đạo hàm bậc một (Gradient) theo η :

$$\nabla_{\eta} \ell(\eta) = \sum_{i=1}^n \left[T(y^{(i)}) - \frac{\partial a(\eta)}{\partial \eta} \right] = \sum_{i=1}^n [T(y^{(i)}) - \mathbb{E}[y | \eta]]$$

Tiếp tục lấy đạo hàm bậc hai để thu được ma trận Hessian:

$$H = \nabla_{\eta}^2 \ell(\eta) = - \sum_{i=1}^n \frac{\partial^2 a(\eta)}{\partial \eta \partial \eta^T} = - \sum_{i=1}^n \text{Var}(y | \eta)$$

Vì giá trị phương sai luôn là một số không âm (và ma trận hiệp phương sai luôn bán xác định dương), ta có:

$$\text{Var}(y | \eta) \succeq 0 \implies H \preceq 0$$

Do ma trận Hessian của hàm Log-Likelihood luôn bán xác định âm, hàm số này **chắc chắn là hàm lõm (Concave)**. Hệ quả là, hàm số đối ngẫu **Negative Log-Likelihood sẽ luôn là hàm lồi (Convex)**. Điều này đảm bảo mọi bài toán tối ưu hóa MLE thuộc Họ phân phối số mũ đều có một điểm cực trị tối ưu toàn cục duy nhất.

4. Khung Mô Hình Tuyến Tính Tổng Quát (Generalized Linear Models - GLMs)

Mô hình tuyến tính tổng quát (GLMs) là phương pháp chuẩn hóa toán học cho phép chúng ta mở rộng ranh giới dự đoán tuyến tính sang các dạng phân phối nhân dị biệt khác nhau. Một mô hình được cấu thành từ 3 giả định nền tảng (Assumptions) sau:

1. **Phân phối mục tiêu:** Biến mục tiêu y cho trước điều kiện biến đầu vào $\mathbf{x}$ phải tuân theo một phân phối xác suất thuộc Họ phân phối số mũ (EFD) với tham số tự nhiên η :

$$y | \mathbf{x}; \beta \sim \text{Exponential Family}(\eta)$$

2. **Thành phần Tuyến tính:** Tham số tự nhiên η được giả định biến thiên tuyến tính theo các đặc trưng đầu vào thông qua tổ hợp trọng số β :

$$\eta = \beta^T \mathbf{x}$$

3. **Hàm liên kết (Link Function):** Kỳ vọng của mô hình $\mu = \mathbb{E}[y | \mathbf{x}]$ liên kết với thành phần tuyến tính thông qua một hàm số g được gọi là hàm liên kết:

$$\eta = g(\mu) = \beta^T \mathbf{x} \implies \mu = g^{-1}(\beta^T \mathbf{x})$$

Nếu ta chọn $\eta = g(\mu)$ sao cho hàm liên kết khớp trực tiếp với định nghĩa tham số tự nhiên, hàm đó được gọi là **Canonical Link Function**.

5. Bản đồ ánh xạ hệ thống GLM

Bảng dưới đây tổng hợp cách khung lý thuyết GLM kiến tạo nên các thuật toán Machine Learning quen thuộc từ các giả định phân phối khác nhau:

Phân phối toán học	Tham số tự nhiên η	Canonical Link Function $g(\mu)$	Hàm dự đoán Inverse Link $g^{-1}(\beta^T \mathbf{x})$	Thuật toán ML cấu thành
Gaussian ($\sigma^2 = 1$)	$\eta = \mu$	Identity link: $\eta = \mu$	$\hat{y} = \beta^T \mathbf{x}$	Linear Regression
Bernoulli	$\eta = \log\left(\frac{\phi}{1-\phi}\right)$	Logit link: $\eta = \log\left(\frac{\mu}{1-\mu}\right)$	$\hat{y} = \frac{1}{1+e^{-\beta^T \mathbf{x}}} = \sigma(\beta^T \mathbf{x})$	Logistic Regression

6. Giai đoạn Dự đoán và Thuật toán Tối ưu hóa Tổng quát

Giai đoạn Dự đoán (Testing Phase)

Tại giai đoạn kiểm thử với một mẫu dữ liệu mới $\mathbf{x}$, mục tiêu của chúng ta là đưa ra kỳ vọng có điều kiện của nhãn y :

$$f_{\beta}(\mathbf{x}) = \mathbb{E}[y | \mathbf{x}; \beta] = \left. \frac{\partial a(\eta)}{\partial \eta} \right|_{\eta=\beta^T \mathbf{x}} = g^{-1}(\beta^T \mathbf{x})$$

Thuật toán Cập nhật Trọng số Tổng quát (Gradient Descent)

Một điều kỳ diệu mang tính hệ thống của GLM đó là: Nếu chúng ta sử dụng *Canonical Link Function*, thì đạo hàm của hàm mất mát Negative Log-Likelihood với mọi phân phối thuộc họ EFD đều cho ra cùng một dạng toán học duy nhất!

Công thức cập nhật vector trọng số tổng quát qua Gradient Descent là:

$$\beta^{(t+1)} = \beta^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) \mathbf{x}^{(i)}$$

Viết chi tiết cho từng thành phần chỉ số thuộc tính β_j :

$$\beta_j^{(t+1)} = \beta_j^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - g^{-1}(\beta^T \mathbf{x}^{(i)})) x_j^{(i)}$$

Nhận xét tối thượng: Cho dù bạn đang làm Linear Regression với phân phối chuẩn, Logistic Regression với phân phối nhị phân, hay thậm chí Poisson Regression với dữ liệu đếm, quy tắc cập nhật trọng số luôn luôn không đổi. Toàn bộ sự khác biệt về mặt bản chất mô hình đã được gói gọn hoàn hảo bên trong hàm ngược liên kết g^{-1} .

7. Thực hành Minh họa bằng Code Python

Dưới đây là đoạn mã Python giúp bạn mô phỏng lại cách một mô hình GLM (như Logistic Regression) tự động kích hoạt hàm liên kết dựa trên phân phối Bernoulli để tối ưu hóa bộ trọng số:

```
import numpy as np
import matplotlib.pyplot as plt

class GLMBernoulliLogistic:
    def __init__(self, learning_rate=0.1, iters=1000):
        self.lr = learning_rate
        self.iters = iters
        self.weights = None

    def _inverse_link_function(self, z):
        # Hàm ngược liên kết ( $g^{-1}$ ) cho phân phối Bernoulli chính là hàm
        # ↪ Sigmoid
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)

        # Vòng lặp tối ưu hóa Gradient Descent tổng quát của hệ thống GLM
        for _ in range(self.iters):
            # 1. Thành phần tuyến tính:  $\eta = \beta^T \cdot x$ 
            eta = np.dot(X, self.weights)

            # 2. Kỳ vọng mô hình qua hàm ngược liên kết:  $\mu = g^{-1}(\eta)$ 
            y_pred = self._inverse_link_function(eta)

            # 3. Cập nhật trọng số theo quy tắc tổng quát của GLM
            gradient = np.dot(X.T, (y - y_pred)) / n_samples
```

```

        self.weights += self.lr * gradient

    def predict_prob(self, X):
        eta = np.dot(X, self.weights)
        return self._inverse_link_function(eta)

# Giả lập dữ liệu nhị phân để chạy thử nghiệm mô hình
np.random.seed(42)
X_dummy = np.random.randn(100, 2)
# Tạo nhãn tuyến tính giả lập bị ép qua sigmoid
true_eta = 1.5 * X_dummy[:, 0] - 2.0 * X_dummy[:, 1]
y_dummy = (1 / (1 + np.exp(-true_eta)) > 0.5).astype(int)

# Khởi tạo và huấn luyện mô hình GLM chuẩn tắc
model = GLMBernoulliLogistic(learning_rate=0.5, iters=500)
model.fit(X_dummy, y_dummy)
print("Bộ trọng số tối ưu tìm được qua GLM:", model.weights)

xx, yy = np.meshgrid(np.linspace(-3, 3, 160), np.linspace(-3, 3, 160))
grid = np.c_[xx.ravel(), yy.ravel()]
probs = model.predict_prob(grid).reshape(xx.shape)

plt.figure(figsize=(7, 5))
contour = plt.contourf(xx, yy, probs, levels=20, cmap="RdBu_r", alpha=0.75)
plt.colorbar(contour, label="$P(y=1 \mid x)$")
plt.scatter(X_dummy[y_dummy == 0, 0], X_dummy[y_dummy == 0, 1], label="class
↪ 0", edgecolor="black", alpha=0.85)
plt.scatter(X_dummy[y_dummy == 1, 0], X_dummy[y_dummy == 1, 1], label="class
↪ 1", edgecolor="black", alpha=0.85)
plt.contour(xx, yy, probs, levels=[0.5], colors="black", linewidths=2)
plt.title("Bernoulli GLM probability surface")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
plt.legend()
plt.grid(alpha=0.2)
plt.show()

```

Bộ trọng số tối ưu tìm được qua GLM: [4.74278036 -5.98881054]

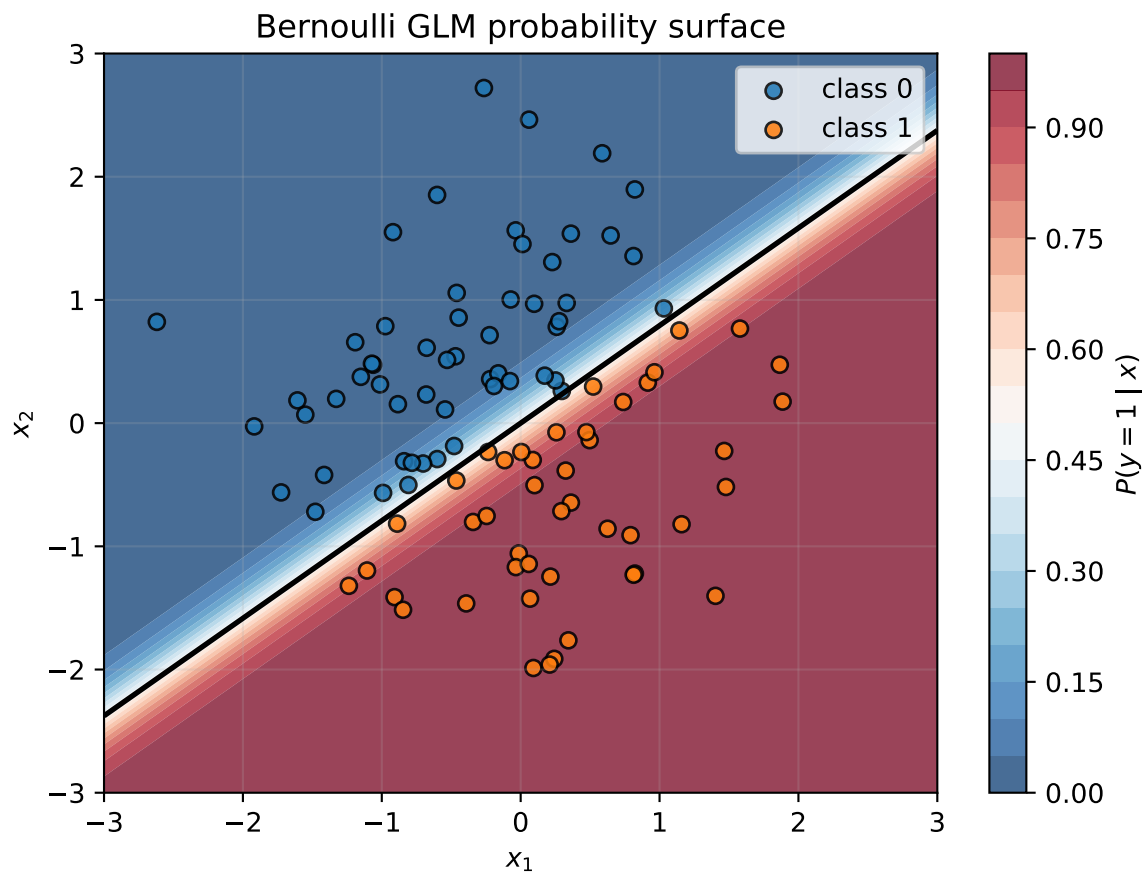


Figure 1: A Bernoulli GLM learns a logistic probability surface through the inverse link function.